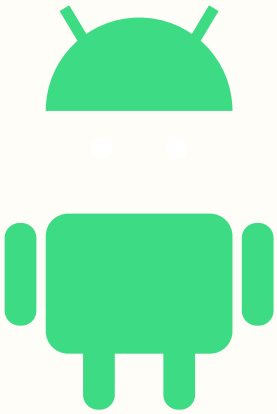
A brain-friendly guide for Android developers

Your Flutter Project, Decoded

Every folder and file explained, mapped to the Android world you already know.



what you know





what you'll learn

INSIDE:

- The big picture: 1 Activity, 1 engine, many widgets
- Every folder & file, with its Android twin
- Inside lib/: how this sample app is organised
- The Android → Flutter Rosetta Stone
- Dart for Kotlin developers, in 15 minutes
- Daily workflow, tools & commands, exercises

You already know 70% of Flutter. You just don't know the new names yet!

How to use this guide

Your brain is picky. It keeps what feels **important**, **surprising** or **emotional**, and it throws away page 47 of a dry reference manual. So this guide is built to be *brain-friendly*: lots of pictures, side-by-side comparisons with the Android stuff you already know, questions that make you think, and a few jokes.

Here's the deal:



BULLET POINTS

- **Don't skip the exercises.** "Sharpen your pencil" boxes are where learning happens. Answers are at the back.
- **Read the "Brain Power" questions** and actually pause for 10 seconds.
- **Keep the sample app open.** Every idea here has a live, animated demo in the app (Learn tab and Components tab).
- **Talk about it.** Explaining Flutter to a colleague (or a rubber duck) moves it to long-term memory.
- **Drink water and sleep.** Seriously - that's when your brain files things away.

1 The Big Picture
Where did all my Activities go?

2 The Project at 10,000 Feet
Every top-level folder, decoded

3 File by File
pubspec.yaml, the android/ folder, and friends

4 Inside lib/
How this sample app is organised - and why

5 The Rosetta Stone
Android concept → Flutter equivalent, one big table

6 Dart for Kotlin Developers
Same ideas, slightly different spelling

7 Your Daily Workflow
Commands, tools, and adding your first screen

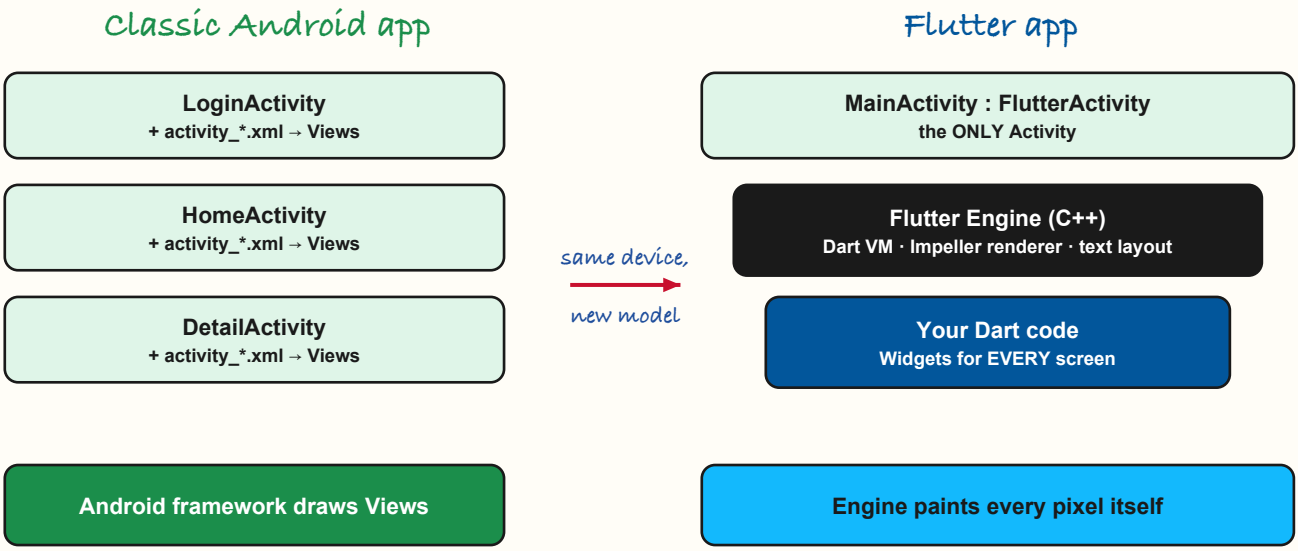
CHAPTER 1

The Big Picture

Wait... where did all my Activities go?

In a classic Android app you have **many Activities** (or one Activity with many Fragments), each inflating XML layouts into real `View` objects that Android draws.

A Flutter app is radically simpler from Android's point of view: there is **exactly one Activity** - `MainActivity`, which extends `FlutterActivity`. It hosts a single surface, and the **Flutter engine paints every pixel itself**. Android never sees a Button or a TextView. It just sees one big canvas.



Think of `FlutterActivity` as a **game engine window**. Games don't use Android Buttons either - they draw everything on a surface. Flutter does the same, but with a beautiful UI toolkit on top.

The layers, from your code down to the metal

Your app (lib/)
Widgets, state, repositories - 100% Dart
← *you work here 99% of the time*

Framework: Material / Cupertino
Buttons, AppBar, Dialogs... ≈ Material Components library

Framework: Widgets · Rendering · Animation · Gestures
Layout, painting, hit testing (all written in Dart!)
← *you can read its source!*

Engine (C++)
Dart runtime, Impeller/Skia graphics, text, platform channels
← *≈ the Android framework*

Embedder: Android (FlutterActivity)
Surface, input events, lifecycle, plugins
← *≈ Activity / Window*

The entire UI framework is Dart source code you can Ctrl+Click into. Try it on "ElevatedButton" - it's just widgets all the way down!



BRAIN POWER

If the engine draws everything itself, what happens to Android features like accessibility (TalkBack), autofill and the soft keyboard? (Hint: the embedder bridges them. Flutter builds a semantics tree for TalkBack.)

there are no Dumb Questions

Q: **So is Flutter a WebView?**

A: No! There is no HTML and no browser. Your Dart code is compiled **ahead-of-time to native ARM machine code** in release builds (it lives in `libapp.so` inside the APK), and the engine renders with the GPU directly.

Q: **Can I still write Kotlin?**

A: Yes. `android/app/src/main/kotlin/.../MainActivity.kt` is a normal Kotlin file. You add native code there and talk to it through **platform channels**. This sample app does exactly that to read the battery level.

Q: **Does my app still have an AndroidManifest.xml?**

A: Yes - permissions, app label, icon, intent filters and deep links still live there. Flutter did not replace Android; it sits on top of it.

CHAPTER 2

The Project at 10,000 Feet

Every top-level folder, decoded

Here is the root of the sample project. Read the handwritten notes - each one tells you what the Android equivalent is.

PROJECT ROOT

```
SampleFlutterApp/
├── lib/
│   └── main.dart
├── android/
├── ios/
├── test/
├── assets/images/
├── docs/
├── pubspec.yaml
├── pubspec.lock
├── analysis_options.yaml
├── .dart_tool/
├── build/
├── .gitignore
└── README.md
```

← the root ≈ your Android Studio project root
← ≈ app/src/main/java + res/layout. ALL your Dart code
← entry point ≈ Application + launcher Activity
← a real Gradle project that hosts Flutter
← Xcode project for iPhone (same idea as android/)
← ≈ src/test + src/androidTest (widget tests)
← ≈ res/drawable + assets/
← these PDFs live here
← ≈ build.gradle dependencies + versionName
← ≈ gradle.lockfile (exact resolved versions)
← ≈ lint.xml / detekt config
← ≈ .gradle/ caches - never edit, never commit
← ≈ app/build/ outputs (APK, AAB) - never commit
← same as Android
← how to run the project

The four folders you'll actually touch

Flutter folder	Android twin	What goes there
lib/	app/src/main/java/ + res/layout/ + res/values/	Every screen, widget, model, service. Most of your day.
test/	app/src/test/ and app/src/androidTest/	Unit + widget tests. Runs in seconds on your laptop - no emulator.
assets/	res/drawable/, res/raw/, assets/	Images, fonts, JSON. Must be declared in pubspec.yaml.
android/	The whole Android project	Manifest, Gradle, signing, native Kotlin. Touch it for permissions, icons, signing, channels.



Watch it!

Folders like `build/`, `.dart_tool/`, `android/.gradle`, `android/local.properties` and `*.iml` files are generated. Never commit them, and never hand-edit them. If things get weird, `flutter clean` deletes them (≈ Build → Clean Project).



SHARPEN YOUR PENCIL

Without looking back: where would you put each of these in a Flutter project?

Thing	Folder / file
A new "ProfileScreen"	
The app's logo PNG	
The CAMERA permission	
A new dependency on "dio"	
A test for your CartProvider	

CHAPTER 3

File by File

pubspec.yaml, the android/ folder, and friends

pubspec.yaml - the heart of the project

If you remember one file, remember this one. It's `build.gradle` (dependencies), part of `AndroidManifest.xml` (version) and the resource system (assets, fonts) rolled into a single YAML file.

pubspec.yaml

```yaml
name: flutter_learning_app                  ← package name used in imports
description: "A learn-by-example..."
publish_to: 'none'                          ← don't publish to pub.dev by accident

version: 1.0.0+1                            ← versionName 1.0.0 + versionCode 1 !!

environment:
  sdk: ^3.5.0                               ← ≈ minimum Kotlin/AGP version

dependencies:                               ← ≈ implementation(...)
  flutter:
    sdk: flutter
  provider: ^6.1.2                          ← ≈ ViewModel + LiveData + Hilt-lite
  http: ^1.2.2                              ← ≈ Retrofit / OkHttp
  shared_preferences: ^2.3.2                ← ≈ SharedPreferences

dev_dependencies:                           ← ≈ testImplementation(...)
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0                     ← ≈ Android Lint rules

flutter:
  uses-material-design: true                ← bundles Material icons font
  assets:                                   ← ≈ res/drawable, but declared
    - assets/images/                        ← whole folder included
```

there are no **Dumb Questions**

Q: **What does the ^ in "^6.1.2" mean?**

A: "Any version compatible with 6.1.2" - i.e. `>=6.1.2 <7.0.0` (semantic versioning). The exact version picked is written into `pubspec.lock`. Commit the lock file for apps so every machine and CI builds the same thing.

Q: **Where do packages come from?**

A: **pub.dev** - the Maven Central of Dart. Run `flutter pub add dio` and it edits pubspec.yaml for you, then downloads (≈ Gradle sync).

Q: **Where is the equivalent of R.drawable.logo?**

A: There's no generated R class. You reference assets by path: `Image.asset('assets/images/logo.png')`. Packages like `flutter_gen` can generate type-safe accessors if you miss R.

The android/ folder - a real Android project in disguise

Open `android/` in Android Studio and you'll feel at home. It is a normal Gradle project whose only job is to **host the Flutter engine and your compiled Dart code**.

android/

```
android/
├── build.gradle.kts                          ← root Gradle script (repos)
├── settings.gradle.kts                       ← applies the Flutter Gradle plugin
├── gradle.properties                         ← JVM args, AndroidX flags
├── gradle/wrapper/                           ← Gradle version pin
├── local.properties                          ← SDK paths - generated, don't commit
├── key.properties                            ← YOUR signing secrets - NEVER commit
└── app/
    ├── build.gradle.kts                      ← applicationId, minSdk, signing, R8
    ├── proguard-rules.pro                    ← R8 keep rules for Kotlin side
    └── src/
        ├── debug/AndroidManifest.xml         ← adds INTERNET for debug
        ├── profile/AndroidManifest.xml
        └── main/
            ├── AndroidManifest.xml           ← permissions, label, deep links
            ├── kotlin/.../MainActivity.kt    ← the ONE Activity + channels
            └── res/
                ├── mipmap-*/ic_launcher.png  ← app icon, as usual
                ├── drawable/launch_background.xml ← splash screen
                ├── values/styles.xml         ← LaunchTheme / NormalTheme
                └── xml/network_security_config.xml ← HTTPS-only (see Doc 4)
```



Watch it!

The template adds the `INTERNET` permission only to the **debug** and **profile** manifests (so hot reload can talk to your computer). If your app uses the network, you must add it to `src/main/AndroidManifest.xml` yourself - otherwise networking works in debug and mysteriously fails in release. This project already has it.

Which values come from where?

Setting	Where it lives	Notes
versionName / versionCode	pubspec.yaml → version: 1.0.0+1	flutter.versionName / flutter.versionCode in app/build.gradle.kts
applicationId	android/app/build.gradle.kts	Set once, never change after publishing
minSdk / targetSdk / compileSdk	flutter.minSdkVersion etc. (defaults)	Override in app/build.gradle.kts if a plugin needs higher
App name on launcher	android:label in AndroidManifest.xml	"Flutter for Android Devs" in this project
App icon	res/mipmap-*	Use flutter_launcher_icons package to generate
Splash screen	res/drawable/launch_background.xml + LaunchTheme	Or use flutter_native_splash package

MainActivity.kt - short, but important

Freshly generated, it's literally one line: `class MainActivity : FlutterActivity()`. In this project we added a **MethodChannel** so Dart can ask Android for the battery level and device info:

android/app/src/main/kotlin/.../MainActivity.kt

```
class MainActivity : FlutterActivity() {
  override fun configureFlutterEngine(
      flutterEngine: FlutterEngine) {
    super.configureFlutterEngine(flutterEngine)
    MethodChannel(flutterEngine.dartExecutor
        .binaryMessenger, "learn.flutter/device")
      .setMethodCallHandler { call, result ->
        when (call.method) {
          "getBatteryLevel" ->
              result.success(batteryLevel())
          else -> result.notImplemented()
        }
      }
  }
}
```

← the only Activity
← called once engine is ready

← registers plugins - keep it!

← name must match Dart side

← reply goes back to Dart

The other root files

File	Android twin	Purpose
analysis_options.yaml	lint.xml / detekt.yml	Which lint rules run in the IDE and in "flutter analyze".
pubspec.lock	gradle.lockfile	Exact versions resolved. Commit it for apps.
test/widget_test.dart	Espresso / Robolectric tests	Pumps widgets in a fake environment. Runs in ms.
ios/	n/a	Runner.xcodeproj, Info.plist (≈ manifest), AppDelegate.swift (≈ MainActivity).
.metadata	n/a	Tracks the Flutter version used to create the project. Commit it.
*.iml, .idea/	*.iml, .idea/	IDE files. Usually ignored.

CHAPTER 4

Inside lib/

How this sample app is organised - and why

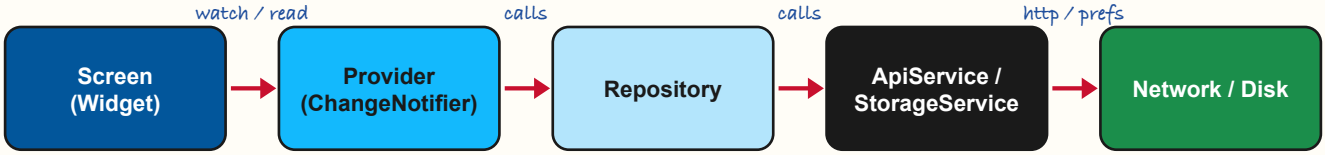
Flutter doesn't force a folder structure on you (Android doesn't really either). This app uses a popular, scalable layout: **core** (app-wide setup), **data** (models, APIs, storage), **state** (ViewModel-like classes), **features** (one folder per area of the app) and **widgets** (reusable custom views).

lib/

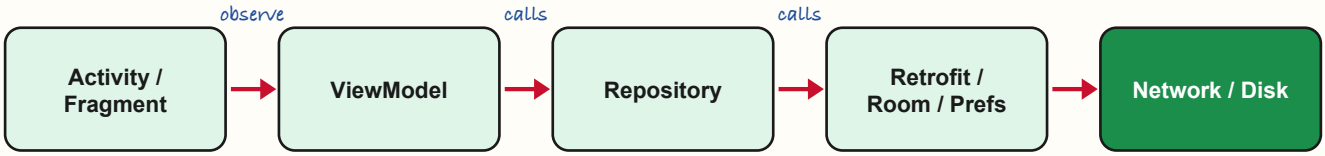
```
lib/
├── main.dart                                  ← main() ≈ Application.onCreate + launcher
├── app.dart                                   ← MaterialApp ≈ tag: theme, routes
├── core/
│   ├── routes/app_routes.dart                 ← screen registry ≈ list + Intents
│   └── theme/app_theme.dart                   ← ≈ themes.xml + colors.xml
├── data/
│   ├── models/post.dart                       ← ≈ data class + Gson
│   ├── services/api_service.dart              ← ≈ Retrofit interface + OkHttp
│   ├── services/storage_service.dart          ← ≈ SharedPreferences wrapper
│   └── repositories/post_repository.dart      ← ≈ Repository (same pattern!)
├── state/
│   ├── theme_provider.dart                    ← ≈ ViewModel + LiveData
│   └── cart_provider.dart                     ← ≈ shared activityViewModel
├── widgets/
│   ├── code_block.dart                        ← reusable "custom views"
│   ├── android_compare_card.dart
│   └── demo_scaffold.dart                     ← ≈ a BaseActivity layout
└── features/                                  ← one folder per feature ≈ feature modules
    ├── home/home_shell.dart                   ← BottomNav + Drawer ≈ MainActivity + Fragments
    ├── learn/                                 ← the concept explainer
    │   ├── concepts_data.dart                 ← all teaching content
    │   ├── learn_screen.dart                  ← search + list ≈ RecyclerView
    │   ├── concept_detail_screen.dart         ← tabs ≈ TabLayout + ViewPager2
    │   ├── concept_demos.dart                 ← picks the animated demo
    │   └── demos/*.dart                       ← 16 animated explainers
    ├── components/*_demo.dart                 ← 13 hands-on component demos
    └── settings/settings_screen.dart          ← ≈ PreferenceFragmentCompat
```

How the pieces talk to each other

It's the **same architecture Google recommends for Android**: UI observes state, state calls a repository, repository calls data sources. Only the names change.



Flutter: UI → state → repository → data sources



Android: the same picture with different names

Follow one tap through the code

Let's trace what happens when you open **Components → Networking**. Keep the files open side by side - this is the fastest way to learn a codebase.

#	File	Flutter code	Android equivalent
1	`components_screen.dart`	onTap → Navigator.pushNamed(context, AppRoutes.network)	startActivity(Intent(this, NetworkActivity::class.java))
2	`core/routes/app_routes.dart`	onGenerateRoute matches "/components/network" → NetworkDemo()	The system resolves the Intent to an Activity
3	`network_demo.dart`	initState(): _future = _repo.getPosts()	ViewModel init { load() }
4	`post_repository.dart`	returns cache or calls ApiService	Repository.getPosts()
5	`api_service.dart`	http.get(...) → jsonDecode → Post.fromJson	Retrofit + Gson converter
6	`network_demo.dart`	FutureBuilder shows spinner, error+Retry, or ListView	observe(Resource) → show loading/error/data

Screen-by-screen map of the sample app

Screen	Flutter pieces used	Android equivalent
HomeShell	NavigationBar + NavigationDrawer + IndexedStack	BottomNavigationView + DrawerLayout + Fragments
LearnScreen	SearchBar + ChoiceChip + SliverList.builder	SearchView + ChipGroup + RecyclerView
ConceptDetailScreen	DefaultTabController + TabBar + TabBarView + Hero	TabLayout + ViewPager2 + shared element
LayoutsDemo	Row, Column, Stack, Expanded, Wrap, LayoutBuilder	LinearLayout, FrameLayout, weight, Flexbox, sw600dp
ListsDemo	ListView.separated, GridView, Dismissible, RefreshIndicator	RecyclerView, GridLayoutManager, ItemTouchHelper, SwipeRefreshLayout
NavigationDemo	pushNamed(arguments) + await result + pop(result)	Intent extras + ActivityResultLauncher

Screen	Flutter pieces used	Android equivalent
FormsDemo	Form, TextFormField, Checkbox, Switch, Slider, pickers	TextInputLayout, CheckBox, Switch, SeekBar, DatePicker
DialogsDemo	showDialog, showModalBottomSheet, SnackBar, PopupMenuButton	AlertDialog, BottomSheetDialogFragment, Toast, PopupMenu
NetworkDemo	http + FutureBuilder + RefreshIndicator	Retrofit + LiveData + SwipeRefreshLayout
StorageDemo	shared_preferences	SharedPreferences / DataStore
StateDemo	ChangeNotifierProvider, Consumer, watch/read	Shared ViewModel + LiveData
StreamsDemo	Stream.periodic, StreamController.broadcast, StreamBuilder	Flow, SharedFlow, collectAsState
LifecycleDemo	State callbacks + AppLifecycleListener	Activity callbacks + ProcessLifecycleOwner
BackgroundDemo	Isolate.run, compute, Timer.periodic	Dispatchers.Default, Handler.postDelayed
AnimationsDemo	AnimatedX, AnimationController, Hero, PageRouteBuilder	ViewPropertyAnimator, ValueAnimator, transitions
PlatformChannelDemo	MethodChannel → MainActivity.kt	Direct Android SDK calls
SettingsScreen	SegmentedButton + ThemeProvider + showAboutDialog	PreferenceFragment + setDefaultNightMode

Fireside Chats

Tonight's talk: Who really owns the screen?

Android Activity

I used to be the boss. I owned the window, inflated layouts, handled rotation, and everybody called my lifecycle methods.

Sixty times a second? That sounds exhausting. When I get recreated on rotation, developers cry.

Hmph. So what do I do all day now?

Flutter Widget

You still own the window - I just paint inside it. And I'm tiny: I'm an immutable description. Build me, throw me away, build me again 60 times a second. I don't mind.

That's the trick - I'm cheap. The expensive stuff (Elements and RenderObjects) is reused. And Flutter's MainActivity declares configChanges, so you aren't even recreated on rotation anymore!

You host the engine, deliver lifecycle events and permissions, and run native code through channels. Honestly? You're the foundation. I'm just the paint.

CHAPTER 5

The Rosetta Stone

Android concept → Flutter equivalent, one big table

Tear this out (figuratively) and pin it next to your monitor. When you think "how do I do *X* in Flutter?", find *X* in the left column.

Android	Flutter
UI building blocks	
Activity	`A route/screen widget (Scaffold). Only one real Activity exists.`
Fragment	`Just a widget. Compose screens from smaller widgets.`
View / ViewGroup	`Widget (StatelessWidget / StatefulWidget)`
XML layout	`The build() method - layout is code`
findViewById / ViewBinding	`Not needed. Keep data in state; rebuild.`
LinearLayout / FrameLayout	`Row, Column / Stack`
ConstraintLayout	`Row/Column/Stack/Expanded, LayoutBuilder, CustomMultiChildLayout`
RecyclerView + Adapter	`ListView.builder / GridView.builder`
ViewPager2 + TabLayout	`PageView / TabBar + TabBarView`
Toolbar / ActionBar	`AppBar`
BottomNavigationView	`NavigationBar`
DrawerLayout	`Scaffold.drawer + NavigationDrawer`
CoordinatorLayout + collapsing	`CustomScrollView + SliverAppBar`
Toast / Snackbar	`ScaffoldMessenger.showSnackBar`
AlertDialog	`showDialog(AlertDialog)`
WebView	`webview_flutter package`
Canvas / custom onDraw	`CustomPaint + CustomPainter`
Navigation	
startActivity(Intent)	`Navigator.push / pushNamed`
Intent extras	`Constructor args / route arguments`
startActivityForResult	`final result = await Navigator.push(...)`
finish()	`Navigator.pop(context, result)`
Navigation Component / deep links	`go_router package`
onBackPressed	`PopScope`
Architecture & state	
ViewModel	`ChangeNotifier / Cubit / Riverpod Notifier`
LiveData / StateFlow	`ChangeNotifier, ValueNotifier, Stream`
observe()	`context.watch<T>(), Consumer, StreamBuilder`
Hilt / Dagger / Koin	`Provider, Riverpod, get_it`
onSaveInstanceState	`RestorationMixin (or persist state)`
Context	`BuildContext (position in the widget tree)`

Android	Flutter
Async & background	
Coroutines / suspend	`Future + async/await`
Flow	`Stream`
Dispatchers.Default / Thread	`Isolate.run / compute`
Handler.postDelayed	`Future.delayed / Timer`
Service / WorkManager	`workmanager plugin, foreground-task plugins`
BroadcastReceiver	`EventChannel or plugin (e.g. connectivity_plus)`
Data	
Retrofit / OkHttp	`http / dio`
Gson / Moshi	`dart:convert + fromJson, or json_serializable`
SharedPreferences / DataStore	`shared_preferences`
EncryptedSharedPreferences	`flutter_secure_storage`
Room / SQLite	`sqflite, drift, Isar/Hive`
ContentProvider	`Platform channel / plugin`
Resources & build	
strings.xml + values-fr	`ARB files + flutter_localizations`
res/drawable	`assets/ declared in pubspec.yaml`
themes.xml / colors.xml	`ThemeData / ColorScheme`
dp / sp	`logical pixels / textScaler`
build.gradle dependencies	`pubspec.yaml dependencies`
Maven Central	`pub.dev`
BuildConfig.DEBUG	`kDebugMode / kReleaseMode`
productFlavors / BuildConfig fields	`--flavor + --dart-define`
R8 / ProGuard	`--obfuscate --split-debug-info (+ R8 for Kotlin side)`
Tooling & testing	
Logcat / Log.d	`debugPrint / dart:developer log (and flutter logs)`
Layout Inspector / Profiler	`Flutter DevTools (Inspector, Performance, Memory)`
Apply Changes	`Hot reload (r) / hot restart (R)`
JUnit	`package:test (unit tests)`
Espresso / Robolectric	`flutter_test widget tests`
UI Automator / instrumented tests	`integration_test package`



RELAX

You don't need to memorise this table. You'll look things up a dozen times, and then one day you'll notice you haven't looked in a week. That's how it sticks.

CHAPTER 6

Dart for Kotlin Developers

Same ideas, slightly different spelling

Good news: Dart and Kotlin are cousins. Both are statically typed, null-safe, support lambdas, generics, extension methods and async code. Here's the translation guide.

Kotlin	Dart	Note
`val name = "Ana"`	`final name = 'Ana';`	final = assigned once
`const val MAX = 10`	`const max = 10;`	const = compile-time constant
`var count = 0`	`var count = 0;`	semicolons are required
`val x: String? = null`	`String? x;`	sound null safety, like Kotlin
`x?.length ?: 0`	`x?.length ?? 0`	?? instead of ?:
`x!!.length`	`x!.length`	one ! instead of two
`lateinit var`	`late String name;`	
`"Hi $name, ${a + b}"`	`'Hi $name, ${a + b}'`	same interpolation
`fun add(a: Int, b: Int) = a + b`	`int add(int a, int b) => a + b;`	return type first
`fun f(name: String = "x")`	`void f({String name = 'x'})`	{ } = named params
`f(name = "y")`	`f(name: 'y')`	
`listOf(1, 2) / mutableListOf()`	`const [1, 2] / [1, 2]`	lists are mutable by default
`mapOf("a" to 1)`	`{'a': 1}`	
`list.map { it * 2 }`	`list.map((e) => e * 2)`	no implicit "it"
`list.filter { }`	`list.where((e) => ...)`	
`data class User(val n: String)`	`class User { final String n; const User(this.n); }`	or use freezed / records
`Pair(a, b)`	`(a, b) // record`	Dart 3 records
`when (x) { is A -> ... }`	`switch (x) { A() => ... }`	Dart 3 patterns
`sealed class Result`	`sealed class Result {}`	exhaustive switch
`fun String.shout() = ...`	`extension on String { String shout() => ... }`	
`object Singleton`	`class S { S._(); static final i = S._(); }`	or top-level vars
`suspend fun load(): T`	`Future<T> load() async`	
`Flow<T>`	`Stream<T>`	
`interface`	`abstract class / abstract interface class`	every class is an interface
`private (keyword)`	`_name (underscore prefix)`	private to the FILE (library)



Watch it!

Privacy in Dart is **per file**, not per class. `_counter` is visible to everything in the same .dart file and invisible outside it. That's why Flutter code often keeps a widget and its private helper widgets in one file.

A real example from the app, annotated

lib/data/models/post.dart (simplified)

```dart
class Post {
  const Post({
    required this.id,
    required this.title,
  });

  final int id;
  final String title;

  factory Post.fromJson(Map<String, dynamic> json) =>
      Post(
        id: json['id'] as int,
        title: json['title'] as String,
      );
}
```

← ≈ data class Post(
← { } = named parameters
← required = must be passed
← this.x = assign field directly

← ≈ val id: Int

← named constructor ≈ companion fun

← as = cast



SHARPEN YOUR PENCIL

Translate these Kotlin lines into Dart:

Kotlin	Dart
val names = users.filter { it.active }.map { it.name }	
val len = text?.length ?: 0	
suspend fun fetch(): List<Post>	

CHAPTER 7

Your Daily Workflow

Commands, tools, and adding your first screen

The commands you'll type every day

Flutter command	Android equivalent	What it does
`flutter create my_app`	New Project wizard	Scaffold a new project
`flutter pub get`	Gradle sync	Download dependencies
`flutter pub add dio`	Add implementation(...) + sync	Add a package
`flutter pub outdated`	Dependency updates lint	See newer versions
`flutter devices`	adb devices	List emulators & phones
`flutter run`	Run ▸ (debug)	Build, install, attach; r/R for reload/restart
`flutter run --release`	Run release variant	Test real performance
`flutter analyze`	./gradlew lint	Static analysis
`dart format .`	Reformat Code	Official formatter (no style debates!)
`flutter test`	./gradlew test	Unit + widget tests
`flutter build apk / appbundle`	./gradlew assembleRelease / bundleRelease	Release artifacts
`flutter clean`	Build → Clean Project	Delete build/ and caches
`flutter doctor -v`	SDK Manager checks	Diagnose your setup
`flutter logs`	adb logcat	Stream device logs

Running this sample app

TERMINAL

```
# 1. Install Flutter (https://docs.flutter.dev/get-started/install)
#    and the Flutter + Dart plugins for Android Studio or VS Code.
flutter doctor            # fix anything marked with a cross

# 2. From the project folder:
flutter pub get
flutter run               # picks your running emulator / phone

# 3. While it runs:  r = hot reload,  R = hot restart,  q = quit
```

Your tools, translated

Tool	What it is for an Android dev
Android Studio + Flutter plugin	Same IDE you know. Run/debug, Flutter Inspector, hot reload button.
VS Code + Flutter extension	Lighter alternative, very popular with Flutter devs.
Flutter DevTools	≈ Layout Inspector + CPU/Memory Profiler + Network inspector, in a browser tab.
Widget Inspector "Select widget mode"	≈ Layout Inspector: tap on the device, jump to the code.
Performance overlay	≈ "Profile GPU rendering" bars.
Android emulator	Works exactly the same. Flutter uses your existing AVDs.

Recipe: add a new screen to this app in 4 steps

ADD A SCREEN

```
// 1. lib/features/profile/profile_screen.dart          ← new file ≈ new Activity class
import 'package:flutter/material.dart';

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(      ← Scaffold ≈ activity root layout
    appBar: AppBar(title: const Text('Profile')),
    body: const Center(child: Text('Hello!')),
  );
}

// 2. core/routes/app_routes.dart                       ← register ≈ in manifest
static const profile = '/profile';
...  profile => const ProfileScreen(),                 ← add a case to the switch

// 3. anywhere with a BuildContext
Navigator.pushNamed(context, AppRoutes.profile);       ← ≈ startActivity(intent)

// 4. Save the file → hot reload. Done!                 ← no manifest, no rebuild
```

Who does what?

Match each Flutter thing on the left with its job on the right. Draw lines!

`pubspec.lock`	Paints every pixel of your UI
`MainActivity.kt`	Where versionName + versionCode are set
`Flutter engine`	Pins the exact version of every package
`pubspec.yaml`	Holds your signing passwords (never commit!)
`key.properties`	Hosts the engine; home of native Kotlin code
`analysis_options.yaml`	Configures lint rules



BULLET POINTS

- A Flutter app runs inside **one** Android Activity; the engine draws everything.
- `lib/` holds all your Dart code; `android/` is a real Gradle project for native config.
- `pubspec.yaml` ≈ build.gradle dependencies + version + assets.
- Add INTERNET permission to the **main** manifest for release builds.
- The recommended architecture is the same as Android: UI → state → repository → data.
- Dart ≈ Kotlin with different spelling. Privacy is per-file (underscore).
- Hot reload keeps state, hot restart resets it, native changes need a full rebuild.

Exercise Solutions

Chapter 2 - Where does it go?

Thing	Answer
A new "ProfileScreen"	lib/features/profile/profile_screen.dart (+ register in app_routes.dart)
The app's logo PNG	assets/images/logo.png, declared under flutter: assets: in pubspec.yaml
The CAMERA permission	android/app/src/main/AndroidManifest.xml (and ios/Runner/Info.plist)
A new dependency on "dio"	pubspec.yaml → dependencies (or run flutter pub add dio)
A test for CartProvider	test/cart_provider_test.dart

Chapter 6 - Kotlin → Dart

```
final names = users.where((u) => u.active).map((u) => u.name).toList();
final len = text?.length ?? 0;
Future<List<Post>> fetch() async { ... }
```

Chapter 7 - Who does what?

Thing	Job
`pubspec.lock`	Pins the exact version of every package
`MainActivity.kt`	Hosts the engine; home of native Kotlin code
`Flutter engine`	Paints every pixel of your UI
`pubspec.yaml`	Where versionName + versionCode are set
`key.properties`	Holds your signing passwords (never commit!)
`analysis_options.yaml`	Configures lint rules

Next up: Doc 2 - the core concepts (widgets, state, layouts, navigation, async) with the same side-by-side treatment.